

CO2: Assessment Pool-1

1. Perform the following operation in LIFO data structure having an array size of 5. Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop(). Point Peak element of the data structure and its index position with a neat sketch explained in detail.

Solution:

Given,

Stack (LIFO)

Array size = 5

Initial Top = -1

Operations:

(1) Push(45)

(2) Push(22)

(3) Push(11)

(4) Push(77)

(5) Pop()

(6) Push(44)

(7) Push(33)

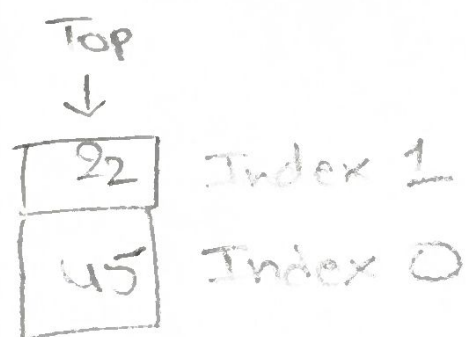
(8) Pop()

(9) Pop()

(10) Pop()

Kishu
27/7/20

<u>S.NO</u>	<u>Operation:</u>	<u>Stack</u>	<u>Top</u>
		45	0
1	Push(45)		1
2	Push(22)	45, 22	2
3	Push(11)	45, 22, 11	3
4	Push(77)	45, 22, 11, 77	2
5	Pop()	45, 22, 11	3
6	Push(44)	45, 22, 11, 44	4
7	Push(33)	45, 22, 11, 44, 33	3
8	Pop()	45, 22, 11, 44	2
9	Pop()	45, 22, 11	2
10	Pop()	45, 22	1



Peak element = 22

Index position = 1

How do you implement Queue data structure using single linked list? with a neat diagram discussing Enqueue and Dequeue operations using SLL.

Solution:

A Queue follows the FIFO (First in First out) Principle. In SLL implementation, 2 pointers are used

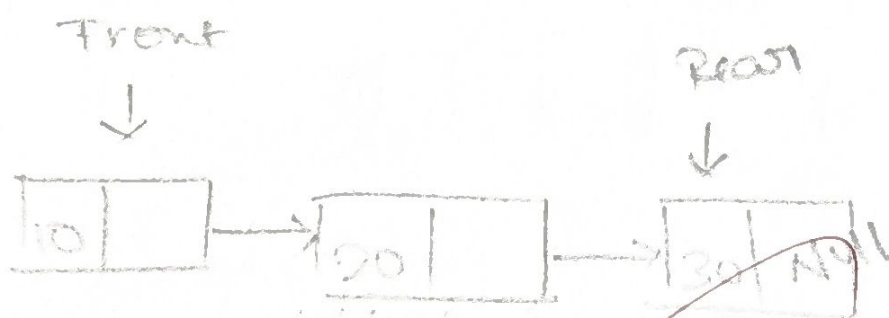
(i) Front $\rightarrow$ Points at first node

(ii) Rear $\rightarrow$ Points at last node

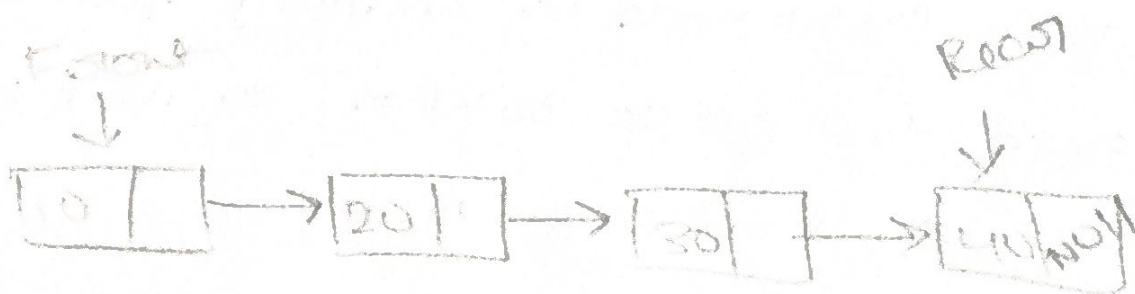
Enqueue operation:

Diagram:

* Before enqueue(uo)



* After enqueue(uo)



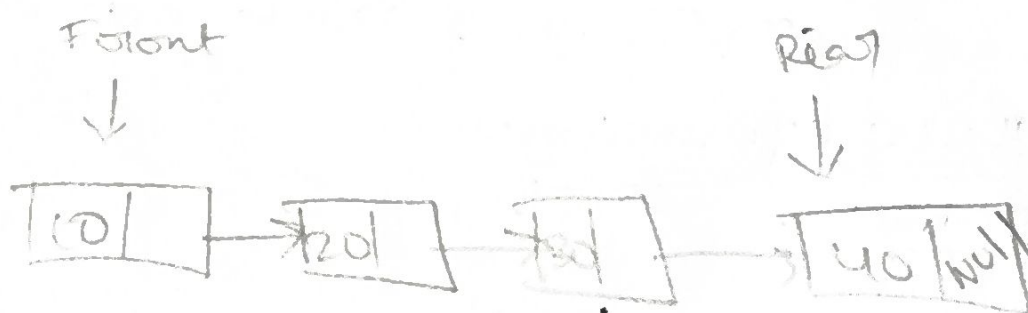
Algorithm:

- 1) Create a node
- 2) Insert data
- 3) Set pointer of new node to NULL
- 4) Link new node to current rear node
- 5) Move Rear to new node
- 6) Stop.

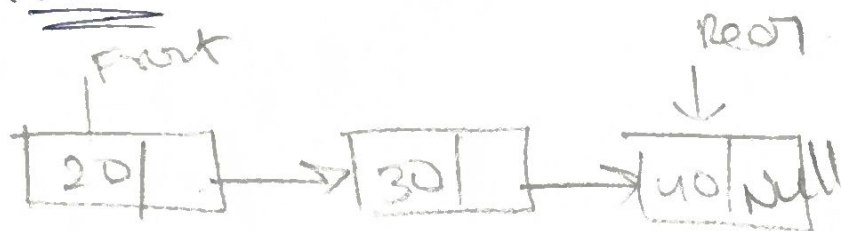
#. Dequeue operation:

Diagram

* Before:



* After:



Algorithm:

- 1) Check whether queue is empty
- 2) If queue is empty, display ~~underflow~~
- 3) Store front node in temporary pointer
- 4) Move the ~~front~~ pointer to next node
- 5) delete temporary node.
- 6) Stop.